Taking a look at the middleware code provided earlier, let's see how we can implement rate limiting, particularly in the context of the Vercel AI SDK.

Listing 9.5 Integrating a rate limiter in middleware

```
import { Ratelimit } from "@upstash/ratelimit";
import { Redis } from "@upstash/redis";

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL,
  token: process.env.UPSTASH_REDIS_REST_TOKEN,
});

const ratelimit = new Ratelimit({
  redis: redis,
  limiter: Ratelimit.slidingWindow(5, "10 s")
});


const rateLimit = async (request, response) => {
  const identifier = request.ip || '127.0.0.1';

  try {
    const { success, limit, reset, remaining } =
await ratelimit.limit(identifier);

    if (!success) {
      console.log("Rate limit exceeded");
      return {
        response: NextResponse.json(
{ message: "Too many requests" }, { status: 429 }),
        continue: false,
      };
    }

    return { continue: true };

  } catch (error) {
    console.error("Rate limiting error:", error);
    return {
      response: NextResponse.json(
{ error: "Internal Server Error" }, { status: 500 }),
      continue: false,
    };
  }
};
```

Imports the rate limit library from upstash

Creates a new Redis client instance

Creates a rate limiter using a sliding window algorithm, allowing 5 requests every 10 seconds

Sets the identifier to the request IP or localhost

Calls the limit method of the Ratelimit instance to check if the client has exceeded the rate limit

The provided code implements rate limiting using Upstash Redis and the `@upstash/ratelimit` library. It's not the only library in existence, but the rate limiter is stable enough to be used in production. One benefit is that Upstash offers a free serverless Redis database with low latency and durable storage for development purposes if you join via their website.